

Build Environment — pqc_crypto_module v0.1.0

Disclaimer: Prepared for FIPS 140-3 evaluation, not currently validated.

1. Rust Toolchain

Component	Version	Notes
Rust edition	2021	Specified in Cargo.toml
Rust toolchain	nightly (required)	Project uses <code>#![feature(unsigned_is_multiple_of)]</code>
Cargo	Bundled with rustup	Package manager and build system
rustfmt	Bundled	Code formatting enforcement
clippy	Bundled	Lint enforcement (<code>-D warnings</code>)

The exact nightly version should be pinned via `rust-toolchain.toml` for reproducible builds. Current recommendation: pin to the nightly version used at the time of lab submission.

2. Target Platforms

Target Triple	Architecture	OS	Status
x86_64-unknown-linux-gnu	x86_64	Linux (glibc)	Primary target for server deployment
aarch64-apple-darwin	ARM64	macOS (Apple Silicon)	Development platform

OS Assumptions: - Linux: Ubuntu 22.04 LTS or later; kernel 5.15+; glibc 2.35+ - macOS: macOS 13 (Ventura) or later; Apple Silicon (M1/M2/M3)

The module delegates entropy to the OS CSPRNG via `getrandom`: - Linux: `getrandom(2)` syscall (kernel 3.17+) - macOS: `SecRandomCopyBytes` / `getentropy(2)`

3. Compiler Flags and Optimization

Setting	Value	Rationale
Optimization (release)	<code>opt-level = 3</code>	Cargo default for release builds
Debug assertions	Disabled in release	Standard Rust behavior
Overflow checks	Disabled in release	Standard Rust behavior; no crypto-relevant integer overflow
LTO	Recommended: <code>lto = true</code>	Link-time optimization for single-binary builds
Codegen units	Recommended: <code>codegen-units = 1</code>	Deterministic compilation
Strip	Recommended: <code>strip = "symbols"</code>	Reduce binary size; not security-relevant

For reproducible builds, the following `Cargo.toml` profile is recommended:

```
[profile.release]
opt-level = 3
lto = true
codegen-units = 1
strip = "symbols"
```

4. Dependency Pinning

Cargo.lock

`Cargo.lock` is committed to the repository. This file pins exact versions of all direct and transitive dependencies, ensuring deterministic builds.

Direct Dependencies

Crate	Version	Purpose
<code>pqcrypto-mldsa</code>	0.1.2	ML-DSA-65 implementation
<code>pqcrypto-traits</code>	0.3	PQC trait definitions
<code>sha3</code>	0.10	SHA3-256 (FIPS 202)
<code>sha2</code>	0.10	SHA-256 (non-approved, legacy)
<code>hmac</code>	0.12	HMAC-SHA256 (non-approved, legacy)
<code>ed25519-dalek</code>	2.1	Ed25519 (non-approved, legacy)

Crate	Version	Purpose
rand	0.8	Random number generation interface
rand_core	0.6	Core RNG traits; getrandom feature
zeroize	1.7	Memory zeroization with derive feature
thiserror	1.0	Error type derivation
hex	0.4	Hex encoding/decoding

Dev Dependencies

Crate	Version	Purpose
tempfile	3.8	Temporary directories for test isolation

5. Deterministic Build Instructions

```
# 1. Pin toolchain (create rust-toolchain.toml if not present)
echo '[toolchain]\nchannel = "nightly-YYYY-MM-DD"' > rust-
    toolchain.toml
```

```
# 2. Verify Cargo.lock is committed
git diff --quiet Cargo.lock || echo "WARNING: Cargo.lock has
    uncommitted changes"
```

```
# 3. Clean build from scratch
cargo clean
```

```
# 4. Build release with deterministic settings
RUSTFLAGS="" cargo build --release -p pqc_crypto_module
```

```
# 5. Compute SHA-256 of output artifact
sha256sum target/release/libpqcrypto_module.rlib
```

```
# 6. Record build metadata
rustc --version --verbose > build-metadata.txt
cargo --version >> build-metadata.txt
uname -a >> build-metadata.txt
date -u >> build-metadata.txt
```

6. Dependency Audit

```
# Scan for known CVEs in dependencies
cargo audit
```

```
# Check for duplicate dependency versions
cargo tree -d
```

```
# Full dependency tree for review
cargo tree -p pqc_crypto_module
```

The following audits should be performed before lab submission: - [x] cargo audit — no known advisories - [] cargo deny check — license and advisory compliance (recommended) - [] Manual review of pqcrypto-mlds source for correctness claims - [] Verify getRandom backend for each target platform

7. Reproducibility Verification

To verify build reproducibility:

1. Build on machine A, record artifact hash.
2. Build on machine B (same toolchain, same Cargo.lock), record artifact hash.
3. Compare hashes. If they differ, investigate codegen-units, LTO settings, and timestamps.

Known limitations: - Rust does not guarantee bit-for-bit reproducible builds across all platforms by default. - codegen-units = 1 and lto = true improve reproducibility. - Path-dependent debug info must be stripped or normalized.